Universidad Señor de Sipán

Taller de Lenguajes de Programación

Sesión 03:

Modelos en Django (ORM)

Introducción

Esta sesión cubre conceptos clave utilizados en Django, un framework web full-stack basado en Python. Si eres principiante en programación, puedes tener dificultades para entender algunos conceptos o términos. Sin embargo, es esencial que empieces a familiarizarte con esos nuevos conceptos o términos. Puedes hojear primero esta sección y volver a este capítulo más adelante, después de leer otros capítulos, que son más tangibles.

En esta sesión introductoria vamos a ver los siguientes temas:

1. Web Framework and Django
2. Websites vs. Django Web Apps
3. How Django Handles HTTP Request and HTTP Response
4. Django's MVT Framework
5. Django Templates vs. Django APIs

Introducción

Esta sesión cubre los modelos de Django, su ORM y la configuración de base de datos. Si completaste la Sesión 2, ya tienes el proyecto de encomiendas corriendo en Docker con PostgreSQL. Ahora profundizaremos en cómo definir correctamente las estructuras de datos, sus relaciones y cómo interactuar con la base de datos desde Python, sin escribir una sola línea de SQL.

En esta sesión cubriremos los siguientes temas:

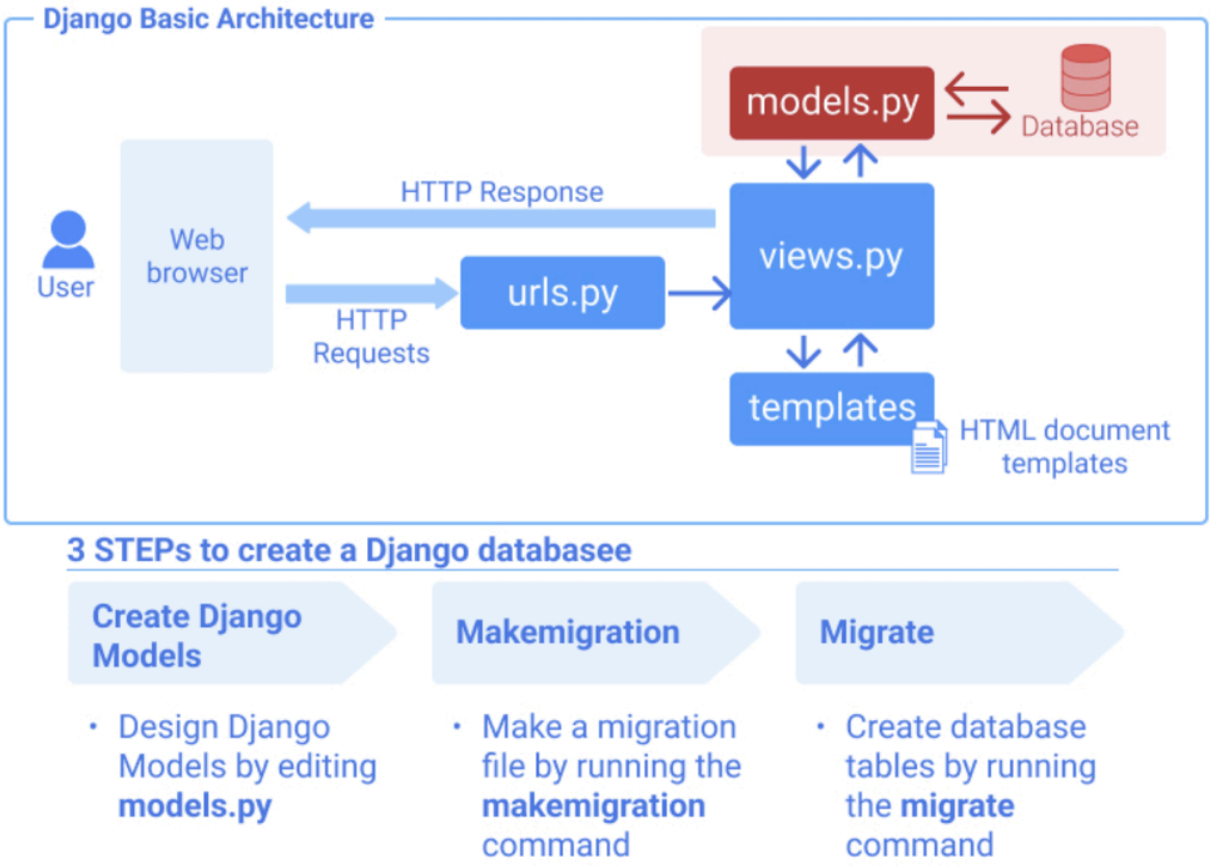
- 3.1. Modelos, Relaciones y Herencia
- 3.2. Introducción al ORM de Django
- 3.3. Configuración de base de datos con variables de entorno
- 3.4. Creación de modelos para el sistema de encomiendas
- 3.5. Migraciones
- 3.6. Relaciones en el ORM: ForeignKey, OneToOne, ManyToMany
- 3.7. Django Shell y consultas ORM

CONTEXTO DEL PROYECTO
El Sistema de Gestión de Encomiendas requiere las siguientes entidades: Cliente, Empleado, Ruta, Encomienda y EstadoEnvio. **Hoy crearemos todos sus modelos, relaciones y ejecutaremos las migraciones dentro del contenedor Docker.**

Modelos, Relaciones y Herencia

Los modelos representan tablas, también conocidas como objetos, dentro de una base de datos. Django ofrece una forma sencilla de mapear objetos a las bases de datos subyacentes de un proyecto. En capítulos posteriores de este libro, utilizaremos este sistema de mapeo para trabajar con otros componentes de Django, como plantillas, vistas o formularios, por nombrar

algunos. Todo lo que dependa del acceso a datos desde una base de datos dependerá de los modelos que creemos.



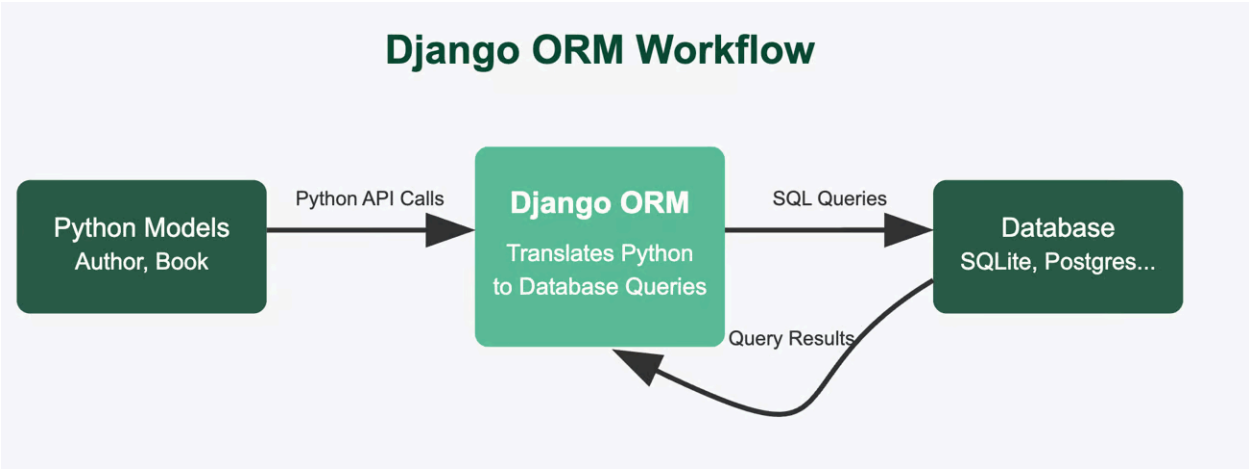
Introducción al ORM de Django

Los modelos representan tablas en una base de datos. Django ofrece un ORM (Object-Relational Mapper) que permite mapear clases Python a tablas de base de datos de forma transparente, sin necesidad de escribir SQL directamente.

Concepto SQL	Equivalente en Django ORM
Tabla	Clase que hereda de models.Model
Columna / Campo	Atributo de clase con tipo de campo (CharField, IntegerField...)
Fila / Registro	Instancia del modelo
Clave primaria	Campo con primary_key=True (Django crea id automático)
Clave foránea	models.ForeignKey(Modelo, on_delete=...)
JOIN	Acceso por atributo o select_related()
SELECT	Modelo.objects.all() / filter() / get()
INSERT	instancia.save() o Modelo.objects.create()
UPDATE	instancia.save() o queryset.update()
DELETE	instancia.delete() o queryset.delete()

VENTAJA CLAVE
Con el ORM de Django nunca escribimos SQL directamente. Los cambios en el modelo se traducen automáticamente en instrucciones SQL para cualquier motor de BD soportado (PostgreSQL, MySQL, SQLite).

Django ofrece una capa de mapeo objeto-relacional (ORM) que te permite interactuar con tu base de datos mediante código Python, en lugar de escribir consultas SQL sin procesar. El ORM proporciona una API de alto nivel, al estilo de Python, para realizar operaciones en la base de datos. Cada modelo es una clase de Python que se hereda de `django.db.models.Model`.



Configurando la base de datos

En la Sesión 2 ya configuramos PostgreSQL dentro de Docker. Repasemos la estructura completa con buenas prácticas, usando variables de entorno para no exponer credenciales en el código fuente.

Archivo .env

Asegúrate de que tu archivo .env en la raíz del proyecto tenga estas variables:

```
None

# .env


# Django

SECRET_KEY=tu-clave-secreta-muy-larga

DEBUG=True

ALLOWED_HOSTS=localhost,127.0.0.1
```

```
# Base de datos PostgreSQL
DB_ENGINE=django.db.backends.postgresql
DB_NAME=encomiendas_db
DB_USER=encomiendas_user
DB_PASSWORD=encomiendas_pass_2026
DB_HOST=db
DB_PORT=5432
```

Configuración en settings.py

Actualiza la sección DATABASES en `config/settings.py` para leer los valores del archivo .env usando python-decouple:

```python
# config/settings.py
from decouple import config

SECRET_KEY = config('SECRET_KEY')
DEBUG = config('DEBUG', cast=bool, default=False)
ALLOWED_HOSTS = config('ALLOWED_HOSTS', default='').split(',')

DATABASES = {
    'default': {
        'ENGINE':   config('DB_ENGINE'),
        'NAME':     config('DB_NAME'),
```

```
        'USER':     config('DB_USER'),
        'PASSWORD': config('DB_PASSWORD'),
        'HOST':     config('DB_HOST', default='localhost'),
        'PORT':     config('DB_PORT', default='5432'),
    }
}

# Idioma y zona horaria para Perú
LANGUAGE_CODE = 'es-pe'
TIME_ZONE = 'America/Lima'
USE_TZ = True
```

Configuración en docker-compose.yml

Recuerda que el servicio PostgreSQL en docker-compose.yml toma las variables del mismo .env:

```XML
version: '3.9'
services:
  web:
    build: .
    command: python manage.py runserver 0.0.0.0:8000
    volumes:
      - .:/app
    ports:
```

```
      - "8000:8000"
    env_file:
      - .env
    depends_on:
      - db

  db:
    image: postgres:15-alpine
    volumes:
      - postgres_data:/var/lib/postgresql/data/
    environment:
      POSTGRES_DB:       ${DB_NAME}
      POSTGRES_USER:     ${DB_USER}
      POSTGRES_PASSWORD: ${DB_PASSWORD}

volumes:
  postgres_data:
```

VERIFICAR CONEXIÓN

Para verificar que Django conecta correctamente a PostgreSQL dentro del contenedor:

docker compose exec web python manage.py dbshell

Si ves el prompt de psql (\#) la conexión está funcionando. Escribe \q para salir.

Creación de Modelos

Los modelos definen la estructura de nuestra base de datos. Para el Sistema de Encomiendas diseñaremos las siguientes entidades y sus atributos.

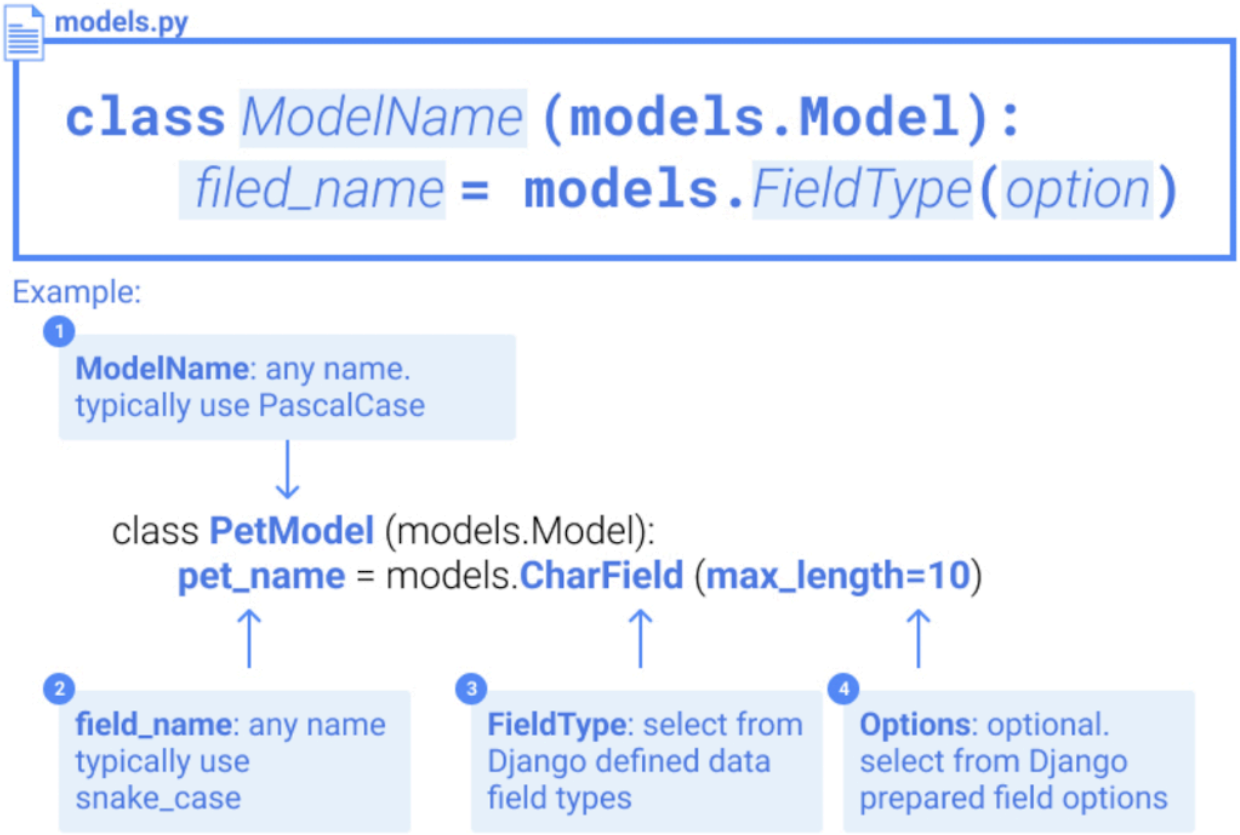


Diagrama de entidades — Sistema de Encomiendas

Entidad	Tabla en BD	Descripción
Cliente	clientes_cliente	Persona que envía o recibe una encomienda
Empleado	envios_empleado	Trabajador que gestiona y registra envíos
Ruta	rutas_ruta	Trayecto desde origen hasta destino

Encomienda	envios_encomienda	Paquete registrado en el sistema
HistorialEstado	envios_historialestado	Registro de cambios de estado de una encomienda

Lab 3.1 — Preparar las apps

Lab 3.1	**Estructura de apps para encomiendas** Verifica que las tres apps estén creadas y registradas en settings.py

```
None

docker compose exec web python manage.py startapp envios

docker compose exec web python manage.py startapp clientes

docker compose exec web python manage.py startapp rutas .
```

Ahora debemos registrar las aplicaciones creadas en *config/settings.py*

```
None

INSTALLED_APPS = [

    'django.contrib.admin',

    'django.contrib.auth',

    'django.contrib.contenttypes',

    'django.contrib.sessions',

    'django.contrib.messages',

    'django.contrib.staticfiles',
```

```
    # Apps del proyecto encomiendas
    'envios',
    'clientes',
    'rutas',
]
```

Archivo choices.py — Enumerados del sistema

Siguiendo el patrón del proyecto de referencia, crearemos un archivo `config/choices.py` con los enumerados del sistema de encomiendas:

```python
# config/choices.py
from django.db import models

class EstadoGeneral(models.IntegerChoices):
    ACTIVO  = 1, 'Activo'
    DE_BAJA = 9, 'De baja'

class EstadoEnvio(models.TextChoices):
    PENDIENTE   = 'PE', 'Pendiente'
    EN_TRANSITO = 'TR', 'En tránsito'
    EN_DESTINO  = 'DE', 'En destino'
    ENTREGADO   = 'EN', 'Entregado'
    DEVUELTO    = 'DV', 'Devuelto'

class TipoDocumento(models.TextChoices):
```

```
    DNI      = 'DNI', 'DNI'
    RUC      = 'RUC', 'RUC'
    PASAPORTE = 'PAS', 'Pasaporte'
```

Modelo Cliente

Edita clientes/models.py y crea el modelo:

```python
# clientes/models.py
from django.db import models
from config.choices import EstadoGeneral, TipoDocumento


class Cliente(models.Model):
        tipo_doc      = models.CharField(
        max_length=3,
        choices=TipoDocumento.choices,
        default=TipoDocumento.DNI
        )
        nro_doc       = models.CharField(max_length=15, unique=True)
        nombres       = models.CharField(max_length=100)
        apellidos    = models.CharField(max_length=100)
        telefono      = models.CharField(max_length=15, blank=True, null=True)
        email         = models.EmailField(blank=True, null=True)
        direccion    = models.TextField(blank=True, null=True)
```

```
    estado       = models.IntegerField(
    choices=EstadoGeneral.choices,
    default=EstadoGeneral.ACTIVO
    )
    fecha_registro = models.DateTimeField(auto_now_add=True)

    def __str__(self):
    return f'{self.nro_doc} - {self.apellidos}, {self.nombres}'

    class Meta:
    db_table     = 'clientes'
    verbose_name  = 'Cliente'
    verbose_name_plural = 'Clientes'
    ordering     = ['apellidos', 'nombres']
```

Modelo Ruta

Edita rutas/models.py:

```python
# rutas/models.py
from django.db import models
from config.choices import EstadoGeneral
```

```
class Ruta(models.Model):
    codigo       = models.CharField(max_length=10, unique=True)
    origen       = models.CharField(max_length=100)
    destino      = models.CharField(max_length=100)
    descripcion = models.TextField(blank=True, null=True)
    precio_base = models.DecimalField(max_digits=10, decimal_places=2)
    dias_entrega = models.PositiveIntegerField(default=1)
    estado       = models.IntegerField(
    choices=EstadoGeneral.choices,
    default=EstadoGeneral.ACTIVO
    )

    def __str__(self):
    return f'{self.codigo}: {self.origen} → {self.destino}'

    class Meta:
    db_table     = 'rutas'
    verbose_name = 'Ruta'
    verbose_name_plural = 'Rutas'
    ordering     = ['origen', 'destino']
```

Modelos Empleado y Encomienda

Edita envios/models.py con los modelos principales del sistema:

Python

```python
# envios/models.py

from django.db import models

from config.choices import EstadoGeneral, EstadoEnvio

from clientes.models import Cliente

from rutas.models import Ruta


class Empleado(models.Model):

        codigo        = models.CharField(max_length=10, unique=True)

        nombres       = models.CharField(max_length=100)

        apellidos    = models.CharField(max_length=100)

        cargo         = models.CharField(max_length=80)

        email         = models.EmailField(unique=True)

        telefono      = models.CharField(max_length=15, blank=True, null=True)

        estado        = models.IntegerField(

        choices=EstadoGeneral.choices,

        default=EstadoGeneral.ACTIVO

        )

        fecha_ingreso = models.DateField()


        def __str__(self):

        return f'{self.codigo} - {self.apellidos}, {self.nombres}'


        class Meta:

        db_table = 'empleados'

        verbose_name = 'Empleado'
```

```
        verbose_name_plural = 'Empleados'
        ordering = ['apellidos']



class Encomienda(models.Model):
        # ── Identificación ──────────────────────────────────
        codigo        = models.CharField(max_length=20, unique=True)
        descripcion = models.TextField()
        peso_kg       = models.DecimalField(max_digits=8, decimal_places=2)
        volumen_cm3 = models.DecimalField(max_digits=12, decimal_places=2,
                                          null=True, blank=True)
        # ── Relaciones ──────────────────────────────────────
        remitente   = models.ForeignKey(
        Cliente, on_delete=models.PROTECT,
        related_name='envios_como_remitente'
        )
        destinatario = models.ForeignKey(
        Cliente, on_delete=models.PROTECT,
         related_name='envios_como_destinatario'
        )
        ruta          = models.ForeignKey(
        Ruta, on_delete=models.PROTECT,
        related_name='encomiendas'
        )
        empleado_registro = models.ForeignKey(
```

```
Empleado, on_delete=models.PROTECT,
 related_name='encomiendas_registradas'
)
# ── Estado y fechas ──────────────────────────
estado       = models.CharField(
max_length=2,
choices=EstadoEnvio.choices,
default=EstadoEnvio.PENDIENTE
)
costo_envio = models.DecimalField(max_digits=10, decimal_places=2)
fecha_registro  = models.DateTimeField(auto_now_add=True)
fecha_entrega_est = models.DateField(null=True, blank=True)
fecha_entrega_real = models.DateField(null=True, blank=True)
observaciones   = models.TextField(blank=True, null=True)

def __str__(self):
return f'{self.codigo} [{self.get_estado_display()}]'

class Meta:
db_table     = 'encomiendas'
verbose_name = 'Encomienda'
verbose_name_plural = 'Encomiendas'
ordering     = ['-fecha_registro']
```

```
class HistorialEstado(models.Model):

       encomienda  = models.ForeignKey(

       Encomienda, on_delete=models.CASCADE,

       related_name='historial'

       )

       estado_anterior = models.CharField(max_length=2,
choices=EstadoEnvio.choices)

       estado_nuevo  = models.CharField(max_length=2, choices=EstadoEnvio.choices)

       observacion   = models.TextField(blank=True, null=True)

       empleado      = models.ForeignKey(

       Empleado, on_delete=models.PROTECT,

       related_name='cambios_estado'

       )

       fecha_cambio  = models.DateTimeField(auto_now_add=True)


       def __str__(self):

       return f'{self.encomienda.codigo}:
{self.estado_anterior}→{self.estado_nuevo}'


       class Meta:

       db_table   = 'historial_estados'

       ordering   = ['-fecha_cambio']
```

Clase Meta: opciones más usadas

En Django, la clase Meta es una clase interna especial que se utiliza para proporcionar metadatos al modelo. No se trata de una característica específica de Django, sino de una característica de Python en la que se puede utilizar una clase interna llamada Meta para configurar el comportamiento de una clase, especialmente en marcos como Django o DRF.

La clase Meta es una forma de personalizar el comportamiento de tu modelo sin añadir nuevos campos. Se define dentro de tu clase de modelo de Django y suele incluir elementos como:

- db_table
- Orden de los conjuntos de consultas
- unique_together
- Nombres detallados, verbose_name_plural
- Permisos
- índices
- default_related_name

¿Por qué se llama Meta?

Porque es la abreviatura de «metadatos»: proporciona información sobre el modelo, no los datos reales que contiene.

Objetivo principal

Cuadro de configuración: sirve como contenedor de las instrucciones que utiliza Django para personalizar la tabla de la base de datos del modelo, el orden de los resultados y el aspecto de la interfaz de administración.

Separación de preocupaciones: mantiene los ajustes de configuración separados de los campos de datos reales para evitar conflictos en los nombres de los atributos.

Opciones meta comunes

- **db_table:** Anula el nombre predeterminado de la tabla de la base de datos (que normalmente es label_modelname) con una cadena personalizada.

- **ordering:** Establece un orden de clasificación predeterminado para los resultados de la consulta. Por ejemplo, ordering = ["-created_at"] ordena los elementos de más reciente a más antiguo.
- **verbose_name y verbose_name_plural:** Proporciona nombres en singular y plural legibles para el modelo, que se muestran en la interfaz de administración de Django.
- **abstract = True:** Marca el modelo como una clase base abstracta. Django no creará una tabla de base de datos para este modelo; en su lugar, sus campos son heredados por los modelos hijos.
- **unique_together / constraints:** Garantiza que las combinaciones de varios campos sean únicas en toda la tabla.
- **indexes:** Define índices de base de datos en campos específicos para acelerar el rendimiento de las búsquedas.
- **proxy = True:** Crea un modelo proxy, lo que permite cambiar el comportamiento del modelo a nivel de Python (como añadir métodos) sin modificar su estructura de base de datos subyacente.

Opción	Para qué sirve	Ejemplo
db_table	Nombre real de la tabla en la BD	db_table = 'encomiendas'
ordering	Orden por defecto de los QuerySets	ordering = ['-fecha_registro']
verbose_name	Nombre legible en el Admin	verbose_name = 'Encomienda'
verbose_name_plural	Nombre en plural en el Admin	verbose_name_plural = 'Encomiendas'
unique_together	Combinación de campos que debe ser única	unique_together = [['nro_doc','tipo_doc']]
indexes	Crear índices para acelerar consultas	indexes = [models.Index(fields=['codigo'])]
constraints	Restricciones personalizadas en la BD	CheckConstraint(check=Q(peso_kg__gt=0), ...)

Migraciones en Django

Las migraciones de Django son un sistema de control de versiones para el esquema de tu base de datos. Registran los cambios realizados en tus modelos de Django (como añadir un campo o eliminar un modelo) y los aplican a la base de datos sin necesidad de escribir código SQL manualmente.

Comandos principales

El flujo de trabajo estándar para gestionar los cambios en el esquema consta de cuatro comandos principales:

- ***makemigrations:*** Analiza tus archivos models.py y agrupa los cambios detectados en nuevos archivos de migración.
- ***migrate***: Aplica estos archivos de migración a la base de datos, creando tablas y columnas según sea necesario.
- ***showmigrations***: Enumera todas las migraciones de tu proyecto e indica si se han aplicado (marcadas con [X]) o no ([ ]).
- ***sqlmigrate <nombre_de_la_aplicación> <nombre_de_la_migración>***: Muestra las sentencias SQL sin procesar que se ejecutarán para una migración específica, lo cual resulta útil para la depuración antes de la aplicación.

Tipos de migraciones

- ***Migraciones de esquema:*** Migraciones automáticas generadas por Django que modifican la estructura de la base de datos.
- ***Migraciones de datos***: Migraciones manuales utilizadas para cambiar los datos en sí (por ejemplo, rellenar un nuevo campo con valores). Se crean utilizando el indicador --empty y la operación RunPython.
- ***Migraciones iniciales***: Las primeras migraciones de una aplicación que crean su estructura de tablas inicial, normalmente denominadas *0001_initial.py.*

Lab 3.2 — Crear y aplicar migraciones dentro de Docker

Lab 3.2	**Ejecutar migraciones del sistema de encomiendas** Todos los comandos se corren dentro del contenedor Docker

```
None

# 1. Asegúrate de que los contenedores estén corriendo

docker compose up -d


# 2. Crear los archivos de migración para todas las apps

docker compose exec web python manage.py makemigrations


# También puedes hacerlo por app si lo prefieres:

docker compose exec web python manage.py makemigrations clientes

docker compose exec web python manage.py makemigrations rutas

docker compose exec web python manage.py makemigrations envios


# 3. Ver los archivos generados

docker compose exec web python manage.py showmigrations


# 4. Aplicar las migraciones a PostgreSQL

docker compose exec web python manage.py migrate


# 5. Verificar el SQL generado (informativo)

docker compose exec web python manage.py sqlmigrate envios 0001
```

Al ejecutar makemigrations, la salida esperada es similar a esta:

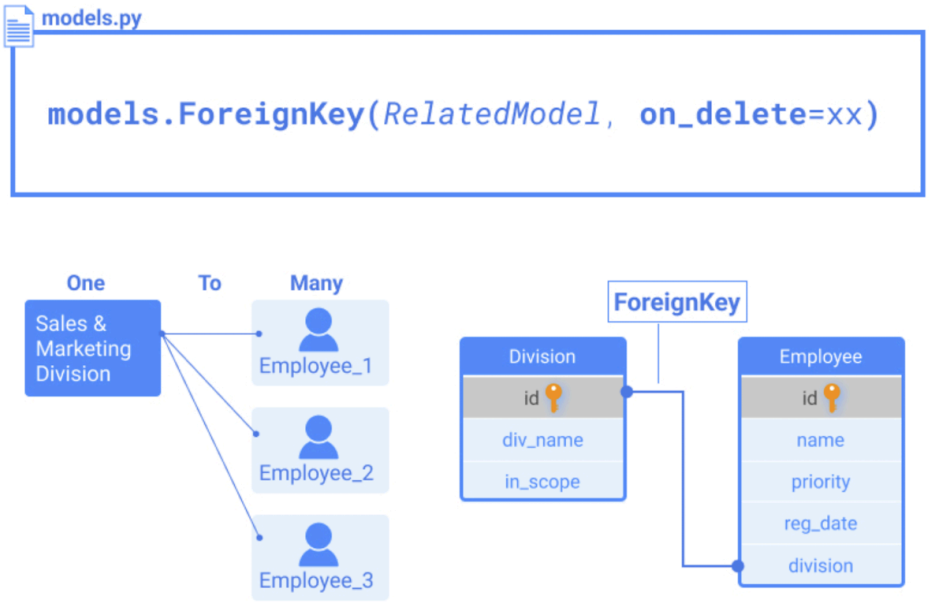
```
None

Migrations for 'clientes':

  clientes/migrations/0001_initial.py

       + Create model Cliente


Migrations for 'rutas':

  rutas/migrations/0001_initial.py

       + Create model Ruta


Migrations for 'envios':

  envios/migrations/0001_initial.py

       + Create model Empleado

       + Create model Encomienda

       + Create model HistorialEstado
```

Relaciones en el ORM Django

Django soporta los tres tipos de relaciones entre modelos en bases de datos relacionales. Cada una tiene un campo específico en el ORM.

ForeignKey — Muchos a uno

Es la relación más común. Muchas encomiendas pueden pertenecer a un mismo cliente remitente.




```
None

# Una encomienda tiene UN remitente (Cliente)

# Un cliente puede ser remitente de MUCHAS encomiendas

remitente = models.ForeignKey(

    Cliente,
```

```
    on_delete=models.PROTECT,        # no borrar el cliente si tiene envios
    related_name='envios_como_remitente'
)
# Uso en el shell:
# Dado un cliente, ver todas sus encomiendas como remitente:
cliente.envios_como_remitente.all()


# Dado una encomienda, ver el cliente remitente:
encomienda.remitente
```

Opciones en on_delete:

- CASCASE: borra el objeto que contiene la ForeignKey.
- PROTECT: previene el borrado del objeto referenciado lanzando ProtectedError
- RESTRICT: previene el borrado del objeto referenciado lanzando RestrictedError
- Como los posibles valores están disponibles en django.db.models, necesitas escribir on_delete=models.CASCADE.

Valor	Comportamiento al borrar el objeto padre
models.CASCADE	**Borra también todos los hijos (encomiendas del cliente)**
models.PROTECT	**Lanza ProtectedError. No permite borrar si tiene hijos**
models.RESTRICT	**Lanza RestrictedError con información más detallada**
models.SET_NULL	**Pone NULL en la FK del hijo (campo debe tener null=True)**
models.SET_DEFAULT	**Pone el valor default en la FK del hijo**
models.DO_NOTHING	**No hace nada. Puede romper la integridad referencial**

RECOMENDACIÓN PARA EL PROYECTO DE ENCOMIENDAS
Usa PROTECT para ForeignKeys de negocio (remitente, empleado, ruta) para evitar borrado accidental de datos. **Usa CASCADE solo para registros que son parte de un todo, como HistorialEstado (si borramos la encomienda, el historial debe borrarse también).**

related_name — la relación inversa

El atributo `related_name` en Django se utiliza para definir el nombre de la relación inversa, es decir, cómo se accede desde el modelo objetivo (el que recibe la clave foránea) hacia el modelo que contiene la relación (ForeignKey, ManyToManyField, o OneToOneField).

Aquí están los puntos clave sobre la relación inversa y `related_name`:

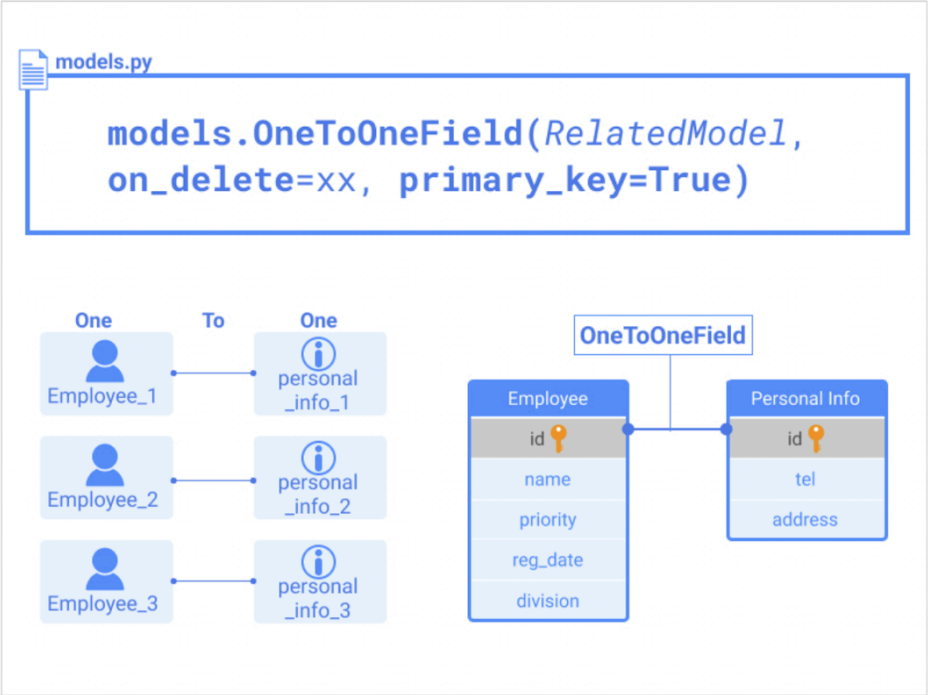
- Comportamiento por defecto: Si no se define un `related_name`, Django crea la relación inversa automáticamente usando el nombre del modelo en minúsculas seguido de `_set` (ej. `nombremodelo_set`).
- Personalización: `related_name` permite especificar un nombre más descriptivo y legible para acceder a los objetos relacionados.
- Uso: Al definir una `ForeignKey(ModelB, related_name='nombre_inverso')` en `ModelA`, puedes acceder a los objetos de `ModelA` desde un objeto de `ModelB` usando `objetoB.nombre_inverso.all()`.
- Ejemplo: Si tienes un modelo `Book` con una clave foránea a `Author`, establecer `related_name='books'` en el campo autor permite usar `author.books.all()` para listar los libros de un autor, en lugar de `author.book_set.all()`.

Importancia: `related_name` es crucial para mejorar la legibilidad del código y evitar conflictos en los nombres de las relaciones inversa

```
None

# Sin related_name: Django generaría 'encomienda_set' para ambas

# -> ERROR: clashes en los reverse accessors


# Con related_name explícito:

remitente    = models.ForeignKey(Cliente, on_delete=models.PROTECT,

                                 related_name='envios_como_remitente')

destinatario = models.ForeignKey(Cliente, on_delete=models.PROTECT,

                                 related_name='envios_como_destinatario')


# Uso desde el shell:

carlos = Cliente.objects.get(nro_doc='12345678')


# Encomiendas donde Carlos es el REMITENTE:

carlos.envios_como_remitente.all()


# Encomiendas donde Carlos es el DESTINATARIO:

carlos.envios_como_destinatario.all()
```

OneToOneField — Uno a uno

Se usa cuando cada instancia de un modelo corresponde exactamente a una instancia de otro. Útil para extender un modelo existente sin modificarlo.

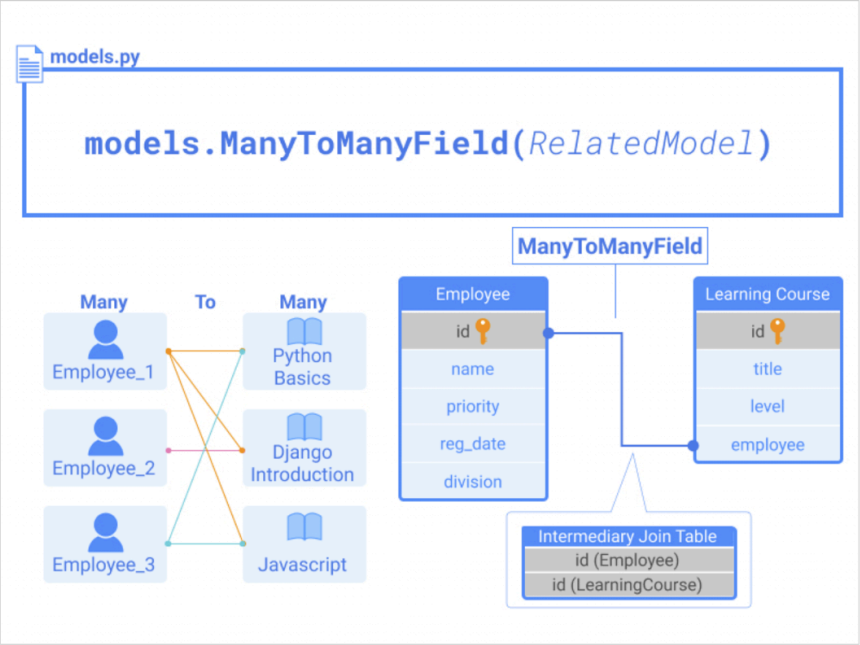



```
None

# Ejemplo: un Empleado puede tener un perfil extendido

class PerfilEmpleado(models.Model):

        empleado      = models.OneToOneField(

        Empleado,

        on_delete=models.CASCADE,

        related_name='perfil'

        )

        foto          = models.ImageField(upload_to='empleados/', null=True,
blank=True)
```

```
    bio           = models.TextField(blank=True)

    linkedin      = models.URLField(blank=True)


    def __str__(self):

    return f'Perfil de {self.empleado}'
```

ManyToManyField — Muchos a muchos

Un empleado puede gestionar varias rutas y una ruta puede tener varios empleados asignados. Django crea automáticamente la tabla intermedia.




```
None

# En el modelo Empleado:
```

```
class Empleado(models.Model):
       # ... campos anteriores ...
       rutas_asignadas = models.ManyToManyField(
       'rutas.Ruta',
       blank=True,
       related_name='empleados_asignados'
       )

# Uso:
emp = Empleado.objects.get(codigo='EMP001')

# Asignar una ruta al empleado
ruta = Ruta.objects.get(codigo='LIM-CHI')
emp.rutas_asignadas.add(ruta)

# Ver todas las rutas de un empleado
emp.rutas_asignadas.all()

# Ver todos los empleados de una ruta
ruta.empleados_asignados.all()

# Quitar una ruta
emp.rutas_asignadas.remove(ruta)
```

Registro en Django Admin

Para administrar los modelos desde el panel de Django Admin, edita envios/admin.py:

```
None

# envios/admin.py

from django.contrib import admin

from .models import Empleado, Encomienda, HistorialEstado


@admin.register(Encomienda)

class EncomiendaAdmin(admin.ModelAdmin):

       list_display  = ('codigo', 'remitente', 'destinatario', 'ruta', 'estado',
'fecha_registro')

       list_filter   = ('estado', 'ruta')

       search_fields = ('codigo', 'remitente__nro_doc', 'destinatario__nro_doc')

       readonly_fields = ('fecha_registro',)


@admin.register(Empleado)

class EmpleadoAdmin(admin.ModelAdmin):

       list_display  = ('codigo', 'apellidos', 'nombres', 'cargo', 'estado')

       search_fields = ('codigo', 'apellidos', 'nombres')


@admin.register(HistorialEstado)

class HistorialEstadoAdmin(admin.ModelAdmin):
```

```
    list_display  = ('encomienda', 'estado_anterior', 'estado_nuevo',
'empleado', 'fecha_cambio')

    readonly_fields = ('fecha_cambio',)
```

Haz lo mismo para clientes/admin.py y rutas/admin.py:

```
None

# clientes/admin.py

from django.contrib import admin

from .models import Cliente


@admin.register(Cliente)

class ClienteAdmin(admin.ModelAdmin):

    list_display  = ('nro_doc', 'tipo_doc', 'apellidos', 'nombres', 'telefono', 'estado')

    list_filter   = ('tipo_doc', 'estado')

    search_fields = ('nro_doc', 'apellidos', 'nombres')
```

```
None

# rutas/admin.py

from django.contrib import admin

from .models import Ruta


@admin.register(Ruta)

class RutaAdmin(admin.ModelAdmin):
```

```
      list_display  = ('codigo', 'origen', 'destino', 'precio_base',
'dias_entrega', 'estado')

      list_filter   = ('estado',)

      search_fields = ('codigo', 'origen', 'destino')
```

Crea el superusuario para acceder al Admin:

```
None

docker compose exec web python manage.py createsuperuser



# Luego accede en: http://localhost:8000/admin
```

Django Shell y Consultas ORM

El shell de Django permite interactuar con los modelos directamente desde la línea de comandos, sin necesidad de crear vistas ni endpoints. Es ideal para realizar consultas y cargar datos de prueba.

Abrimos el terminal y ejecutamos lo siguiente

```
None
Terminal
# Abrir el shell dentro del contenedor Docker

docker compose exec web python manage.py shell

# Salir del shell
>>> exit()
```

Lab 3.3 — Operaciones CRUD desde el shell

Lab 3.3	**Consultas ORM sobre el sistema de encomiendas** Carga datos de prueba y practica las operaciones básicas

Importar los modelos

```
None
# Siempre importa primero los modelos que necesitas
from clientes.models import Cliente
```

```
from rutas.models import Ruta
from envios.models import Empleado, Encomienda, HistorialEstado
from config.choices import EstadoEnvio, EstadoGeneral, TipoDocumento
from datetime import date
```

1. Crear registros

```
None
# Crear un cliente remitente
c1 = Cliente.objects.create(
       tipo_doc='DNI',
       nro_doc='12345678',
       nombres='Carlos',
       apellidos='Ramirez Torres',
       telefono='987654321',
       email='carlos@mail.com'
)

# Crear cliente destinatario
c2 = Cliente.objects.create(
       tipo_doc='DNI',
       nro_doc='87654321',
       nombres='Ana',
       apellidos='Flores Diaz',
       telefono='912345678'
```

```
)

# Crear una ruta
r1 = Ruta.objects.create(
       codigo='LIM-TRU',
       origen='Lima',
       destino='Trujillo',
       precio_base=25.00,
       dias_entrega=2
)

# Crear un empleado
e1 = Empleado.objects.create(
       codigo='EMP001',
       nombres='Luis',
       apellidos='Mendoza Cruz',
       cargo='Operador de envios',
       email='luis@encomiendas.pe',
       fecha_ingreso=date.today()
)

# Crear una encomienda
enc = Encomienda.objects.create(
       codigo='ENC-2026-001',
       descripcion='Caja con ropa y documentos',
       peso_kg=3.5,
       remitente=c1,
```

```
        destinatario=c2,
        ruta=r1,
        empleado_registro=e1,
        costo_envio=r1.precio_base,
        fecha_entrega_est=date(2026, 4, 20)
)
print(enc)  # ENC-2026-001 [Pendiente]
```

2. Recuperar registros

```python
# Todos los registros de un modelo
Encomienda.objects.all()

# Un solo registro (lanza DoesNotExist si no encuentra)
enc = Encomienda.objects.get(codigo='ENC-2026-001')

# Filtrar por estado
Encomienda.objects.filter(estado=EstadoEnvio.PENDIENTE)

# Acceder a objetos relacionados
enc.remitente       # objeto Cliente
enc.remitente.nombres # 'Carlos'
enc.ruta.destino    # 'Trujillo'

# Desde el cliente, ver sus encomiendas
```

```
c1.envios_como_remitente.all()
```

3. Actualizar registros

```python
# Actualizar un campo y guardar
enc = Encomienda.objects.get(codigo='ENC-2026-001')
enc.estado = EstadoEnvio.EN_TRANSITO
enc.save()

# Registrar el cambio en el historial
HistorialEstado.objects.create(
       encomienda=enc,
       estado_anterior=EstadoEnvio.PENDIENTE,
       estado_nuevo=EstadoEnvio.EN_TRANSITO,
       empleado=e1,
       observacion='Encomienda recogida y en camino a destino'
)

# Actualizar múltiples registros con update()
Encomienda.objects.filter(estado=EstadoEnvio.PENDIENTE).update(
       estado=EstadoEnvio.EN_TRANSITO
)
```

4. Lookup expressions — Filtros avanzados

```python
# exact: valor exacto (por defecto en filter)
Encomienda.objects.filter(estado__exact='TR')

# iexact: igual pero case-insensitive
Ruta.objects.filter(destino__iexact='trujillo')

# contains: contiene el texto (case-sensitive)
Encomienda.objects.filter(descripcion__contains='caja')

# icontains: contiene el texto (case-insensitive)
Encomienda.objects.filter(descripcion__icontains='Caja')

# in: el valor está en la lista
Encomienda.objects.filter(estado__in=['PE','TR'])

# gte / lte: mayor o igual / menor o igual
Encomienda.objects.filter(peso_kg__gte=5)
Encomienda.objects.filter(costo_envio__lte=30)

# Consultas con relaciones (doble underscore __)
# Encomiendas cuyo remitente tenga DNI '12345678'
Encomienda.objects.filter(remitente__nro_doc='12345678')

# Encomiendas en la ruta Lima-Trujillo
Encomienda.objects.filter(ruta__codigo='LIM-TRU')
```

5. Eliminar registros

```python
# Eliminar una instancia específica
enc = Encomienda.objects.get(codigo='ENC-2026-001')
enc.delete()

# Eliminar con filtro (cuidado con esto en producción)
Encomienda.objects.filter(estado='DV').delete()
```

Validaciones a nivel de modelo

Las validaciones a nivel de modelo en Django son cruciales para garantizar la integridad de los datos, ya que actúan como la última línea de defensa antes de guardar información en la base de datos, independientemente de cómo se envíen los datos (formularios, APIs, o scripts) Django permite definir reglas de negocio directamente en el modelo para garantizar la integridad de los datos antes de que lleguen a la base de datos. Existen tres niveles de validación que se complementan entre sí.

Nivel	Dónde se define	Cuándo se ejecuta
validators en el campo	Parámetro del Field	Al llamar full_clean() o desde formularios/DRF
clean_fields()	Método del modelo	Valida cada campo de forma individual
clean()	Método del modelo	Valida reglas que cruzan varios campos
save()	Método del modelo	Llama full_clean() explícitamente si se sobreescribe

Validators en el campo

Los validators son funciones que reciben un valor y lanzan ValidationError si no cumple la regla. Puedes usar los que trae Django o crear los propios. Para ello agregamos un un archivo llamado validators en este caso en el app envios (envios/vaidators.py)

```python
# envios/validators.py
from django.core.exceptions import ValidationError
from django.utils import timezone

def validar_peso_positivo(value):
    if value <= 0:
        raise ValidationError(
            f'El peso debe ser mayor a 0. Recibió: {value} kg'
        )

def validar_codigo_encomienda(value):
    """El código debe empezar con ENC-"""
    if not value.startswith('ENC-'):
        raise ValidationError(
            'El código de encomienda debe comenzar con ENC-'
        )

def validar_nro_doc_dni(value):
    """El DNI debe tener exactamente 8 dígitos numéricos"""
    if not value.isdigit() or len(value) != 8:
        raise ValidationError(
```

```
            'El DNI debe contener exactamente 8 dígitos numéricos'
```

Ahora aplicamos estos validators en los campos de los modelos, abrimos el archivo envios/models.py e incluimos esto:

```python
# envios/models.py
from .validators import validar_peso_positivo, validar_codigo_encomienda
from django.core.validators import MinValueValidator, RegexValidator

class Encomienda(models.Model):
    codigo = models.CharField(
        max_length=20,
        unique=True,
        validators=[validar_codigo_encomienda]
    )
    peso_kg = models.DecimalField(
        max_digits=8,
        decimal_places=2,
        validators=[
            validar_peso_positivo,
            MinValueValidator(0.01, message='El peso mínimo es 0.01 kg')
        ]
    )
    costo_envio = models.DecimalField(
        max_digits=10,
        decimal_places=2,
```

```
        validators=[MinValueValidator(0)]
    )
    # ... resto de campos
```

Método clean() — validaciones cruzadas

El método `clean()` se usa para validar reglas que involucran más de un campo a la vez. Por ejemplo: que la fecha de entrega estimada no sea anterior a hoy, o que el remitente y destinatario sean personas distintas:

```python
# envios/models.py
from django.core.exceptions import ValidationError
from django.utils import timezone

class Encomienda(models.Model):
        # ... campos ...

        def clean(self):
        """Validaciones cruzadas entre campos del modelo"""
        errors = {}

        # Regla 1: remitente y destinatario no pueden ser la misma persona
        if self.remitente_id and self.destinatario_id:
                if self.remitente_id == self.destinatario_id:
```

```
        errors['destinatario'] = ValidationError(
            'El destinatario no puede ser el mismo que el remitente.'
        )

    # Regla 2: la fecha de entrega estimada no puede ser en el pasado
    if self.fecha_entrega_est:
        if self.fecha_entrega_est < timezone.now().date():
        errors['fecha_entrega_est'] = ValidationError(
            'La fecha de entrega estimada no puede ser en el pasado.'
        )

    # Regla 3: la fecha de entrega real no puede ser antes de la estimada
    if self.fecha_entrega_est and self.fecha_entrega_real:
        if self.fecha_entrega_real < self.fecha_entrega_est:
        errors['fecha_entrega_real'] = ValidationError(
            'La fecha de entrega real no puede ser antes de la estimada.'
        )

    if errors:
        raise ValidationError(errors)

    def save(self, *args, **kwargs):
    """Ejecutar full_clean() antes de guardar"""
    self.full_clean()   # dispara: clean_fields() + clean()
    super().save(*args, **kwargs)
```

ADVERTENCIA SOBRE SAVE() Y FULL_CLEAN()

Django NO llama full_clean() automáticamente al usar .save() directamente.
Solo se llama cuando el modelo pasa por un formulario de Django o por DRF.
Si sobreescribes save() llamando full_clean(), las validaciones funcionarán también desde el shell y desde la API REST.

Lab 3.4 — Probar validaciones desde el shell

Lab 3.4	**Validaciones en acción** Verifica que las reglas de negocio se aplican correctamente

Python

```python
from envios.models import Encomienda, Empleado
from clientes.models import Cliente
from rutas.models import Ruta
from django.core.exceptions import ValidationError
from datetime import date

# Recuperar objetos existentes
c1 = Cliente.objects.get(nro_doc='12345678')  # Carlos (remitente)
r1 = Ruta.objects.get(codigo='LIM-TRU')
e1 = Empleado.objects.get(codigo='EMP001')

# --- Caso 1: peso inválido ---
try:
```

```
        enc = Encomienda(
        codigo='ENC-2026-002', descripcion='Prueba',
        peso_kg=-1,   # inválido
        remitente=c1, destinatario=c1,   # mismo cliente: también inválido
        ruta=r1, empleado_registro=e1,
        costo_envio=25, fecha_entrega_est=date(2025, 1, 1)  # fecha pasada
        )
        enc.save()
except ValidationError as e:
        print('Errores encontrados:')
        for campo, mensajes in e.message_dict.items():
        print(f'  [{campo}]: {mensajes}')

# Salida esperada:
# Errores encontrados:
#   [peso_kg]: ['El peso debe ser mayor a 0. Recibió: -1 kg']
#   [destinatario]: ['El destinatario no puede ser el mismo que el remitente.']
#   [fecha_entrega_est]: ['La fecha de entrega estimada no puede ser en el
pasado.']
```

Propiedades y Métodos de Negocio

"Fat models, skinny views" es la filosofía central de Django. La lógica de negocio vive en el modelo, no en las vistas ni en los serializers. Esto hace el código más testeable, reutilizable y fácil de mantener.

Tipo	Sintaxis	Para qué sirve
Propiedad	@property	Campo calculado de solo lectura, se usa como atributo
Método de instancia	def metodo(self)	Lógica sobre un objeto específico
Método de clase	@classmethod	Lógica relacionada con la clase, no con una instancia
Método estático	@staticmethod	Utilidades que no necesitan el objeto ni la clase

@property — Campos calculados

Los `@property` se usan para exponer datos derivados de otros campos sin guardarlos en la base de datos. Son ideales para DRF: los serializers pueden incluirlos como si fueran campos normales.

Python

```
clientes/models.py – @property
# clientes/models.py
class Cliente(models.Model):
        nombres   = models.CharField(max_length=100)
        apellidos = models.CharField(max_length=100)
        # ... resto de campos ...
```

```python
    @property
    def nombre_completo(self):
    """Nombre y apellidos en formato legible"""
    return f'{self.apellidos}, {self.nombres}'

    @property
    def esta_activo(self):
    """Devuelve True si el estado es ACTIVO"""
    return self.estado == EstadoGeneral.ACTIVO

    @property
    def total_encomiendas_enviadas(self):
    """Número de encomiendas donde este cliente es remitente"""
    return self.envios_como_remitente.count()
```

envios/models.py – @property

```python
# envios/models.py
from django.utils import timezone

class Encomienda(models.Model):
    # ... campos ...

    @property
    def esta_entregada(self):
    return self.estado == EstadoEnvio.ENTREGADO
```

```
    @property
    def esta_en_transito(self):
    return self.estado == EstadoEnvio.EN_TRANSITO

    @property
    def dias_en_transito(self):
    """Días transcurridos desde el registro"""
    if not self.fecha_registro:
        return 0
    delta = timezone.now().date() - self.fecha_registro.date()
    return delta.days

    @property
    def tiene_retraso(self):
    """True si la fecha estimada ya pasó y no está entregada"""
    if not self.fecha_entrega_est or self.esta_entregada:
        return False
    return timezone.now().date() > self.fecha_entrega_est

    @property
    def descripcion_corta(self):
    """Primeros 50 caracteres de la descripción"""
    return self.descripcion[:50] + '...' if len(self.descripcion) > 50 else
self.descripcion
```

```python
# envios/models.py
from django.utils import timezone

class Encomienda(models.Model):
    # ... campos ...

    @property
    def esta_entregada(self):
    return self.estado == EstadoEnvio.ENTREGADO

    @property
    def esta_en_transito(self):
    return self.estado == EstadoEnvio.EN_TRANSITO

    @property
    def dias_en_transito(self):
    """Días transcurridos desde el registro"""
    if not self.fecha_registro:
        return 0
    delta = timezone.now().date() - self.fecha_registro.date()
    return delta.days

    @property
    def tiene_retraso(self):
    """True si la fecha estimada ya pasó y no está entregada"""
    if not self.fecha_entrega_est or self.esta_entregada:
```

```python
            return False
        return timezone.now().date() > self.fecha_entrega_est


    @property
    def descripcion_corta(self):
    """Primeros 50 caracteres de la descripción"""
    return self.descripcion[:50] + '...' if len(self.descripcion) > 50 else self.descripcion
```

Métodos de instancia — Lógica de negocio

Los métodos de instancia encapsulan acciones sobre un objeto. El caso perfecto en el sistema de encomiendas es el cambio de estado con su historial asociado:

```python
# envios/models.py
class Encomienda(models.Model):
    # ... campos y properties ...


    def cambiar_estado(self, nuevo_estado, empleado, observacion=''):
    """
    Cambia el estado de la encomienda y registra el cambio
    en HistorialEstado automáticamente.
    Uso: enc.cambiar_estado(EstadoEnvio.EN_TRANSITO, empleado=e1)
    """
    if nuevo_estado == self.estado:
```

```
            raise ValueError(f'La encomienda ya se encuentra en estado
{self.get_estado_display()}')

        estado_anterior = self.estado
        self.estado = nuevo_estado

        if nuevo_estado == EstadoEnvio.ENTREGADO:
            from django.utils import timezone
            self.fecha_entrega_real = timezone.now().date()

        self.save()

        # Registrar en el historial
        HistorialEstado.objects.create(
            encomienda=self,
            estado_anterior=estado_anterior,
            estado_nuevo=nuevo_estado,
            empleado=empleado,
            observacion=observacion
        )
        return self

        def calcular_costo(self):
        """
        Calcula el costo del envío basándose en el precio
        base de la ruta y el peso del paquete.
        """
        PRECIO_POR_KG_EXTRA = 2.50  # soles por kg adicional
```

```
PESO_BASE = 5.0            # los primeros 5kg están en el precio base


costo = self.ruta.precio_base
if self.peso_kg > PESO_BASE:
        costo += (self.peso_kg - PESO_BASE) * PRECIO_POR_KG_EXTRA
return round(costo, 2)
```

Métodos de clase — @classmethod

En Django, el decorador `@classmethod` se utiliza para definir métodos dentro de un modelo (o cualquier clase) que actúan sobre la clase misma en lugar de sobre una instancia específica (objeto) del modelo.

A diferencia de los métodos de instancia regulares que reciben `self`, los métodos de clase reciben `cls` como su primer argumento, lo que les permite acceder a la clase, sus atributos y realizar consultas a la base de datos utilizando el gestor del modelo (`cls.objects...`)

Los `@classmethod` reciben la clase como primer argumento (`cls`) y son útiles para crear objetos con lógica especial o hacer consultas frecuentes desde el modelo:

Características clave en Django

- Acceso a la clase (`cls`): Se usa `cls` para referirse a la clase actual.
- No requieren instancia: Se pueden llamar directamente desde el modelo (`MiModelo.metodo_clase()`).
- Ideal para consultas generales:

```python
# envios/models.py
class Encomienda(models.Model):
    # ... campos, properties y métodos ...

    @classmethod
    def crear_con_costo_calculado(cls, remitente, destinatario, ruta,
                                  empleado, descripcion, peso_kg,
                                  **kwargs):
    """
    Fábrica: crea una encomienda calculando el costo automáticamente.
    El llamador no necesita saber la fórmula de precio.
    """
    import uuid
    from django.utils import timezone
    from datetime import timedelta

    codigo = f'ENC-{timezone.now().strftime("%Y%m%d")}-{str(uuid.uuid4())[:6].upper()}'
    fecha_est = timezone.now().date() + timedelta(days=ruta.dias_entrega)

    encomienda = cls(
        codigo=codigo,
        descripcion=descripcion,
        peso_kg=peso_kg,
        remitente=remitente,
        destinatario=destinatario,
        ruta=ruta,
        empleado_registro=empleado,
```

```
            fecha_entrega_est=fecha_est,
            **kwargs
        )
        encomienda.costo_envio = encomienda.calcular_costo()
        encomienda.save()
        return encomienda

# Uso desde el shell o desde una vista:
enc = Encomienda.crear_con_costo_calculado(
        remitente=c1, destinatario=c2, ruta=r1,
        empleado=e1, descripcion='Documentos legales', peso_kg=1.5
)
print(enc.costo_envio)   # 25.00 (precio base, peso < 5kg)
print(enc.codigo)         # ENC-20260418-A3F9B2
```

Probando desde el shell

```python
from envios.models import Encomienda, HistorialEstado
from clientes.models import Cliente
from config.choices import EstadoEnvio

# Recuperar la encomienda y el empleado
enc = Encomienda.objects.get(codigo='ENC-2026-001')
```

```
e1  = Empleado.objects.get(codigo='EMP001')

# --- Usar el método de negocio cambiar_estado ---
enc.cambiar_estado(
       nuevo_estado=EstadoEnvio.EN_TRANSITO,
       empleado=e1,
       observacion='Encomienda recogida en agencia Lima'
)

print(enc.estado)          # TR
print(enc.esta_en_transito)   # True (property)
print(enc.dias_en_transito)   # 0 (recien registrada)
print(enc.tiene_retraso)   # False

# Verificar que el historial se creó automáticamente
enc.historial.all()
# <QuerySet [<HistorialEstado: ENC-2026-001: PE->TR>]>

# --- Usar la propiedad del cliente ---
c1 = enc.remitente
print(c1.nombre_completo)               # 'Ramirez Torres, Carlos'
print(c1.total_encomiendas_enviadas)   # 1
print(c1.esta_activo)                   # True

# --- Crear con código automático ---
c2 = Cliente.objects.get(nro_doc='87654321')
r1 = Ruta.objects.get(codigo='LIM-TRU')
```

```
nueva = Encomienda.crear_con_costo_calculado(
       remitente=c1, destinatario=c2, ruta=r1, empleado=e1,
       descripcion='Zapatos y ropa de niño', peso_kg=7.0
)
print(nueva.codigo)        # ENC-20260418-XXXXXX
print(nueva.costo_envio)  # 30.0  (25 base + 2*2.50 por 2kg extra)
```

Managers y Querysets personalizados

Cada modelo tiene un manager por defecto llamado `objects`. Un Manager personalizado permite agregar métodos de consulta semánticos directamente en el modelo, haciendo el código de las vistas más limpio y expresivo.

Sin Manager personalizado	Con Manager personalizado
Encomienda.objects.filter(estado='PE')	Encomienda.objects.pendientes()
Encomienda.objects.filter(estado__in=['PE','TR'])	Encomienda.objects.activas()
Encomienda.objects.filter(ruta=ruta, estado='TR')	Encomienda.objects.en_transito_por_ruta(ruta)
Cliente.objects.filter(estado=1)	Cliente.objects.activos()

QuerySet personalizado — la base

La forma moderna en Django es definir un `QuerySet` personalizado primero y luego usarlo como Manager. Así los métodos también funcionan en cadena (chaining):

```python
# envios/querysets.py  (crear este archivo nuevo)
from django.db import models


class EncomiendaQuerySet(models.QuerySet):


    # ── Filtros por estado ──────────────────────────
    def pendientes(self):
    return self.filter(estado='PE')
```

```python
    def en_transito(self):
    return self.filter(estado='TR')

    def entregadas(self):
    return self.filter(estado='EN')

    def devueltas(self):
    return self.filter(estado='DV')

    def activas(self):
    """Pendientes + en tránsito + en destino"""
    return self.filter(estado__in=['PE', 'TR', 'DE'])

    # ── Filtros compuestos ──────────────────────────
    def por_ruta(self, ruta):
    return self.filter(ruta=ruta)

    def por_remitente(self, cliente):
    return self.filter(remitente=cliente)

    def por_destinatario(self, cliente):
    return self.filter(destinatario=cliente)

    def en_transito_por_ruta(self, ruta):
    return self.en_transito().por_ruta(ruta)

    # ── Con retraso ────────────────────────────────
```

```
    def con_retraso(self):
    """Encomiendas activas cuya fecha estimada ya pasó"""
    from django.utils import timezone
    return self.activas().filter(
        fecha_entrega_est__lt=timezone.now().date()
    )

    # ── Optimización de consultas ──────────────────
    def con_relaciones(self):
    """Precarga las relaciones más usadas (evita el problema N+1)"""
    return self.select_related(
        'remitente', 'destinatario', 'ruta', 'empleado_registro'
    )


class ClienteQuerySet(models.QuerySet):

    def activos(self):
    return self.filter(estado=1)

    def de_baja(self):
    return self.filter(estado=9)

    def con_dni(self):
    return self.filter(tipo_doc='DNI')

    def buscar(self, termino):
```

```
        """Búsqueda por nombre, apellido o número de documento"""
        return self.filter(
             models.Q(nombres__icontains=termino) |
             models.Q(apellidos__icontains=termino) |
             models.Q(nro_doc__icontains=termino)
        )



class RutaQuerySet(models.QuerySet):

        def activas(self):
        return self.filter(estado=1)

        def por_origen(self, ciudad):
        return self.filter(origen__icontains=ciudad)

        def por_destino(self, ciudad):
        return self.filter(destino__icontains=ciudad)
```

Registrar el Manager en los modelos

Para activar el QuerySet personalizado, reemplaza el Manager por defecto usando `as_manager()` o creando una clase Manager explícita:

```python
# envios/models.py
from .querysets import EncomiendaQuerySet


class Encomienda(models.Model):
    # Reemplaza el manager 'objects' con el personalizado
    objects = EncomiendaQuerySet.as_manager()

    # ... todos los campos, clean(), save(), properties, métodos ...
```

```python
# clientes/models.py
from envios.querysets import ClienteQuerySet


class Cliente(models.Model):
    objects = ClienteQuerySet.as_manager()
    # ... campos ...
```

```python
# clientes/models.py
```

```python
# rutas/models.py
from envios.querysets import RutaQuerySet

class Ruta(models.Model):
    objects = RutaQuerySet.as_manager()
    # ... campos ...
```

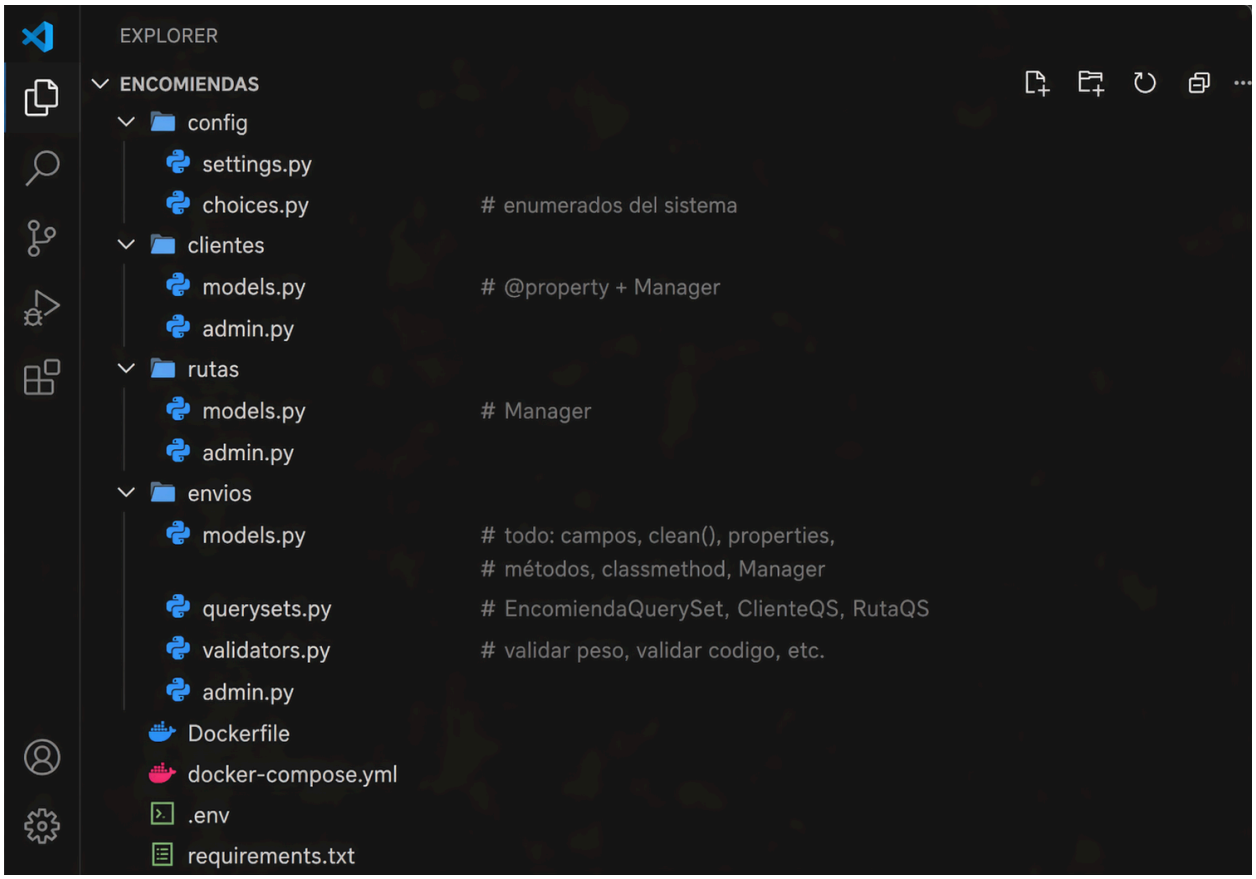
Lab 3.5 — Managers en acción desde el shell

Lab 3.5	**QuerySets personalizados** Compara el código antes y después de usar managers

```python
from envios.models import Encomienda
from clientes.models import Cliente
from rutas.models import Ruta

# ── Queries semánticas ──────────────────────────
Encomienda.objects.pendientes()
Encomienda.objects.en_transito()
Encomienda.objects.con_retraso()
Cliente.objects.activos()
```

```
Ruta.objects.activas().por_origen('Lima')

# ── Encadenamiento (chaining) ──────────────────
ruta = Ruta.objects.get(codigo='LIM-TRU')
cliente = Cliente.objects.get(nro_doc='12345678')

# Encomiendas activas del cliente Carlos en la ruta Lima-Trujillo
Encomienda.objects.activas().por_remitente(cliente).por_ruta(ruta)

# Encomiendas en tránsito en esa ruta (método compuesto)
Encomienda.objects.en_transito_por_ruta(ruta)

# ── Búsqueda de clientes ──────────────────────
Cliente.objects.buscar('rami')   # busca en nombre, apellido y doc
Cliente.objects.activos().con_dni()

# ── Contar y agregar ──────────────────────────
Encomienda.objects.pendientes().count()
Encomienda.objects.con_retraso().count()

# ── Con precarga de relaciones (sin N+1) ────────
# Con esto, acceder a enc.remitente no hace una query adicional
encomiendas = Encomienda.objects.activas().con_relaciones()
for enc in encomiendas:
    print(enc.remitente.nombre_completo, '→', enc.ruta.destino)
```

Estructura final de archivos del proyecto



ENTREGABLE DE LA SESIÓN 3

Al finalizar las semanas 3-5 debes poder demostrar:

Modelos y migraciones

1. Modelos Cliente, Ruta, Empleado, Encomienda e HistorialEstado creados.
2. Archivo config/choices.py con todos los enumerados del sistema.
3. Archivo envios/validators.py con al menos 3 validadores personalizados.
4. Archivo envios/querysets.py con QuerySets para los tres modelos.
5. Migraciones aplicadas y tablas visibles en la BD PostgreSQL (Docker).
6. Modelos registrados en el Admin de Django.

Validaciones

7. El método clean() en Encomienda valida: remitente != destinatario, fecha de entrega no en el pasado y coherencia de fechas.
8. save() llama full_clean() correctamente.
9. Desde el shell: demostrar que un ValidationError se lanza con datos incorrectos.

Propiedades y métodos

10. Cliente tiene: nombre_completo, esta_activo, total_encomiendas_enviadas.
11. Encomienda tiene: esta_entregada, tiene_retraso, dias_en_transito, descripcion_corta.
12. Encomienda tiene el método cambiar_estado() que actualiza e historializa.
13. Encomienda tiene el classmethod crear_con_costo_calculado().

Managers personalizados

14. Encomienda.objects.pendientes() / activas() / con_retraso() funcionan.
15. Cliente.objects.activos() y buscar(termino) funcionan.
16. Encadenamiento: Encomienda.objects.activas().por_ruta(r).count().

Repositorio actualizado en GitHub con todos los archivos nuevos.

Próxima sesión: Componentes de Django y Django REST Framework — crearemos las primeras URLs, Views y Serializers para exponer la API de encomiendas.